

to that particular cluster, check the pod forwarding, and then stream the logs to see if everything is working well or not.

All in all, you will have to keep repeating the same steps as per the number of changes you make and the number of changes you want to look at. So we now need a tool like Skaffold that can detect the activities that are common in nature, and automate and take care of these behind the scenes. Let us understand this from a developer workflow perspective.

Automating common activities

In Figure 2, you can see some key activities marked in green that developers care about – things like coding and configuring the application to make it run. Marked in red are the things that they hope will just get done automatically by a tool to avoid the repetitive work that they are doing as part of the developer workflow. Skaffold allows developers to work on what they care about and takes care of the rest of the things.

Skaffold's end-to-end development pipelines are as follows.

- `$ skaffold dev`: Whenever you make a change to your code, the *dev* cycle automatically gets activated.
- `$ skaffold run`: *Run* allows you to run your particular application.
- `$ skaffold debug`: As the name suggests, *debug* helps in debugging a particular application that you want as a part of this process.

Once Skaffold has sensed the changes being made to the code and to the application's configuration, it will perform all the steps — building the code, filling the containers, pushing it to the registry, applying any configuration deployment and bringing the pods up on its own. It will then stream the logs to you so you can actually see that. It is capable of working with various clusters and registries.

```
$ docker build . -t gcr.io/project-id/myapp:v2
$ docker push gcr.io/project-id/myapp:v2
$ vi myapp-deployment.yaml
$ kubectl apply -f myapp-deployment.yaml
$ kubectl get pods
$ kubectl port-forward some-pod-qx54dw xxxx:yyyy
$ kubectl logs -f some-pod-qx54dw
```

Figure 1: *kubectl* command is initiated

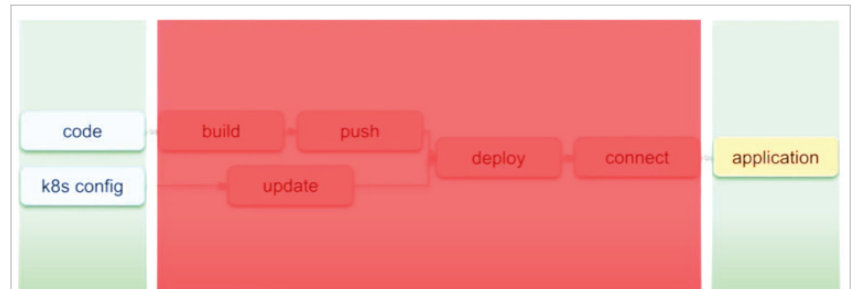


Figure 2: Steps in the development process

```

1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     for {
10        fmt.Println("Hello Skaffold")
11        time.Sleep(time.Second * 1)
12    }
13 }
14
15
```

Figure 3: *main.go*

```

1 FROM golang:1.15 as builder
2 COPY main.go .
3 # 'skaffold debug' sets SKAFFOLD_GO_GCFLAGS to disable compiler opti
4 ARG SKAFFOLD_GO_GCFLAGS
5 RUN go build -gcflags="${SKAFFOLD_GO_GCFLAGS}" -o /app main.go
6
7 FROM alpine:3
8 # Define GOTRACEBACK to mark this container as using the Go language
9 # for 'skaffold debug' (https://skaffold.dev/docs/workflows/debug/).
10 ENV GOTRACEBACK=single
11 CMD ["/app"]
12 COPY --from=builder /app .
  
```

Figure 4: Folder consisting of *Dockerfile*

A demo

Here we have got a very straightforward *go* file (Figure 3). This file contains one main file and one main function, and it is printing out 'Hello Skaffold'.

In the same folder, we have got a *Dockerfile* that you can see in Figure

4. This is a multi-staged *Dockerfile*. We are essentially just creating an executable for this file in the app, and in the next step we are going to set the default command to run when the container is up, which will in turn generate the output file. This is from